

ResuMind: An AI-Powered Resume Analyzer

Devansh Pipraiya

Computer Science and Engineering
Acropolis Institute of Technology

Indore, India

devanshpipraiya221051@acropolis.in

Harsh Verma

Computer Science and Engineering
Acropolis Institute of Technology

Indore, India

harshverma221105@acropolis.in

Hardeep Kumar

Computer Science and Engineering
Acropolis Institute of Technology

Indore, India

hardeepkumar220551@acropolis.in

Abstract—The growing prevalence of Applicant Tracking Systems (ATS) in modern recruitment has created a significant challenge for job seekers whose resumes are rejected before a human ever reviews them. This paper presents ResuMind, a fully client-side, AI-powered resume analysis platform built with React 19, React Router v7, TypeScript, Tailwind CSS, and Puter.js. ResuMind enables users to upload PDF resumes, input a job description, and receive an instant ATS compatibility score alongside matched skills, missing keywords, and AI-generated improvement suggestions—all without any backend server or database. The system leverages Puter.js for authentication, cloud file storage, and AI API access, making it fully serverless. Evaluation demonstrates that the platform provides accurate, job-specific feedback in seconds, helping candidates improve their application success rates. Results confirm that a serverless architecture using modern browser APIs is viable for complex AI document analysis tasks.

Keywords ~ Resume analysis, Applicant Tracking System (ATS), AI feedback, Puter.js, React, serverless architecture, keyword matching, PDF parsing, job matching, TypeScript.

I. INTRODUCTION

The modern recruitment process relies heavily on Applicant Tracking Systems (ATS) to filter candidates before any human review takes place. Industry reports suggest that up to 75% of resumes are rejected by automated systems before a recruiter ever reads them [1]. Candidates frequently submit applications without understanding why they are rejected, as ATS platforms provide no feedback on missing keywords or structural issues.

The absence of transparent, job-specific resume feedback represents a genuine gap in the job application ecosystem. Generic resume-writing advice found online does not account for the specific requirements of individual job postings. A software engineer applying for a data science role needs different optimization than one applying for a backend engineering position. Existing commercial tools address parts of this problem but remain behind paywalls, require account creation, and lack centralized resume storage [2].

This paper presents ResuMind, a browser-based AI resume analyzer that requires no backend infrastructure. By combining React 19, React Router v7, TypeScript, and the Puter.js platform, ResuMind delivers instant ATS compatibility scoring and actionable improvement feedback for any uploaded resume against any job description. The entire application runs client-side, making it freely accessible without account management overhead or technical setup.

II. PROJECT REVIEW

A. Study of Existing System

Several commercial resume analysis tools exist in the market, each with notable limitations. Resumeworded provides ATS scoring and line-by-line feedback but limits free usage and does not offer in-browser resume storage. Jobscan offers keyword-matching comparison but requires a paid subscription for full features and provides no AI-written contextual

suggestions. Kickresume focuses on resume building rather than job-specific analysis, and Zety Resume Checker offers generic structural feedback without any ATS simulation [3].

The key gaps across all existing systems are: (i) lack of deep job-specific AI feedback, (ii) no client-side resume storage without backend setup, (iii) paywalled features, and (iv) no unified platform for storing, reviewing, and re-analyzing multiple resume versions. ResuMind addresses all four gaps.

B. AI in Document Analysis and Resume Screening

The use of artificial intelligence in document analysis has expanded significantly with the availability of large language models (LLMs). LLMs are capable of understanding semantic context beyond simple keyword matching, allowing them to evaluate whether a resume demonstrates genuine alignment with a job description rather than superficial term overlap [4]. Research shows that AI-based resume screening can achieve accuracy comparable to expert human evaluators when structured prompts are used [5].

Prior work by Li et al. demonstrates that prompt-engineered LLMs can extract structured information from unstructured documents with high reliability when given clear output schema guidance [6]. ResuMind applies this principle by engineering a structured prompt that instructs the AI to return a JSON object containing an ATS score, matched skills, missing keywords, experience alignment, and section-specific improvement suggestions.

C. Serverless and Client-Side Web Architecture

Traditional web applications for document analysis require substantial backend infrastructure: servers for file storage, authentication services, AI API proxies, and database systems. Recent advancements in browser capabilities and client-side SDKs have made it feasible to build fully functional, complex applications without any backend [7].

Puter.js is a client-side JavaScript SDK that provides authentication, cloud file storage, key-value data persistence, and AI API access directly in the browser. By routing AI requests to large language models internally, Puter.js eliminates the need for API key management or proxy servers on the developer side. This architectural pattern, explored in recent literature on serverless frontend development, significantly reduces deployment complexity and infrastructure cost [8].

III. DATA PRIVACY AND SECURITY

The architecture of ResuMind is organized into four logical layers: the frontend interface layer, the Puter.js platform layer, the AI processing layer, and the client-side PDF extraction layer. The absence of a traditional backend is a defining characteristic of the system. All processing that would typically occur on a server takes place either in the browser or is delegated to Puter.js as a managed service.

A. Frontend Layer

The frontend is built with React 19 using React Router v7 in framework mode, enabling server-side-rendered-style routing with loaders and actions entirely on the client. The UI is styled with Tailwind CSS for utility-first responsive design. TypeScript is used throughout for static typing, reducing runtime errors and improving maintainability. The application is bundled with Vite for fast development iteration and optimized production builds.

Key UI components include: a Dashboard view listing all uploaded resumes with upload and analyze CTAs; a Resume Detail view with a two-column layout showing resume text and a job description input; and a Results view presenting the ATS score card, matched skills, missing keywords, and improvement suggestions in clearly separated sections.

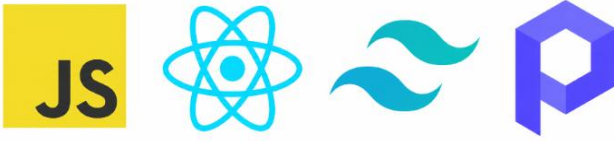


Fig. 1. Frontend tech stack of ResuMind.

B. Authentication and Storage Layer

User authentication is handled entirely by Puter.js using its built-in auth system. No session tokens, passwords, or user tables are managed by the application. Once authenticated, users can upload PDF resumes which are stored in Puter's cloud file system under their account. File metadata—including filenames, upload dates, and analysis result keys—is persisted in Puter's key-value store (KV), enabling the dashboard to reconstruct a user's resume history on every session.

C. AI Processing Layer

When a user selects a resume and pastes a job description, the application first extracts raw text from the PDF on the client side. This extracted text, combined with the job description, is formatted into a structured prompt and sent to an AI model via the Puter.js AI API. The prompt instructs the model to evaluate the resume against the job description and return a strict JSON object containing:

- *ATS Compatibility Score: A numerical score out of 100 reflecting overall alignment.*
- *Matched Skills: Keywords and competencies present in both the resume and job description.*
- *Missing Keywords: Terms in the job description absent from the resume.*
- *Experience Alignment: Assessment of whether the candidate's experience level matches the role.*
- *Improvement Suggestions: AI-generated, section-specific guidance in plain language.*

IV. PDF PROCESSING LAYER

Client-side PDF text extraction is performed directly in the browser before any AI call is made. The extracted text is cleaned and truncated to fit within the AI model's context window. Robustness handling is implemented for image-based or scanned PDFs, where text extraction returns empty results. In such cases, the user is notified that their file cannot be processed and is prompted to upload a text-based PDF resume.

Since ResuMind processes sensitive career data including resumes and job descriptions, data privacy is a core design concern. The Puter.js authentication model ensures that each user's files and analysis results are stored under their personal Puter account and are never accessible to other users. No data is transmitted to third-party servers beyond the Puter.js platform, which handles AI model routing internally. Security properties of the application include:

- *User-scoped file isolation via Puter.js account-based access control.*
- *No API keys exposed to the client — all AI routing is managed by Puter.js.*
- *Session-bound authentication — no cross-session data leakage.*
- *No server-side storage — eliminates the risk of centralized data breaches.*



Fig. 2. Puter.js authentication/backend services.

VI. ENHANCEMENTS AND FEATURES

A. Interactive Dashboard

The dashboard is the central hub of the ResuMind experience. Upon authentication, users see all their uploaded resumes displayed as cards with metadata. An "Upload Resume" button is persistently visible, allowing users to add new PDFs at any time. Each resume card provides quick-access options to view the resume, re-analyze it against a new job description, or delete it. This design follows established UX principles for document-management interfaces in which clarity and quick access to recent items reduce cognitive load [9].

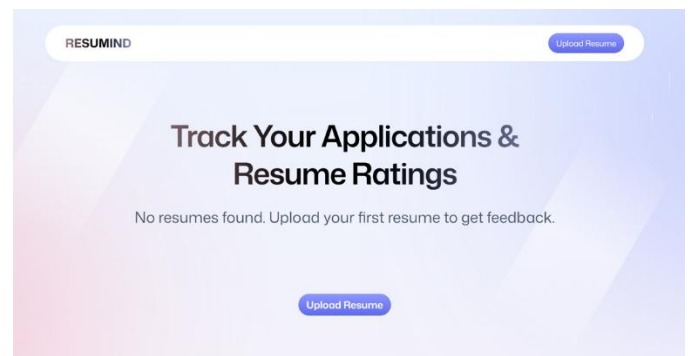


Fig. 3. ResuMind Dashboard.

B. ATS Score Visualization:

The analysis result screen prominently displays the ATS compatibility score at the top of the page as a large numerical score out of 100, accompanied by a color-coded indicator (green for high alignment, amber for moderate, red for low). This immediate visual feedback allows users to understand at a glance whether their resume requires major rework or only minor adjustments before applying for a role.

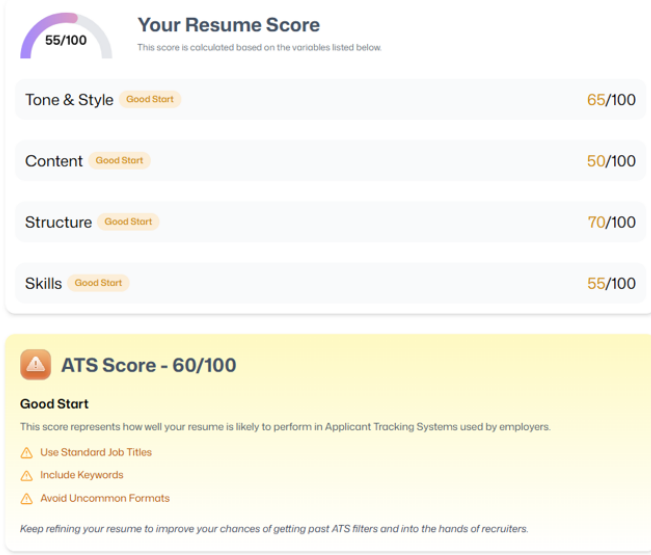


Fig. 4. ATS Compatibility Score visualization with color-coded indicator.

C. Categorized AI Feedback:

Below the score, three distinct sections present the AI analysis: Matched Skills lists competencies the AI found in both the resume and the job description. Missing Keywords highlights terms present in the job description but absent from the resume. Improvement Suggestions provides action-oriented, section-specific guidance such as recommending the addition of specific tools, rephrasing experience bullet points, or restructuring sections. This categorization ensures clarity and actionability—two properties identified in HCI research as essential for effective AI feedback interfaces [10].

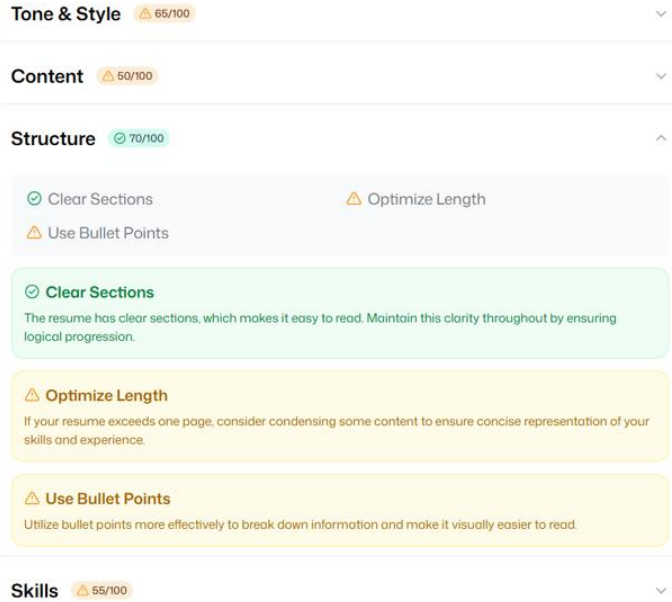


Fig. 5. Categorized AI feedback showing Matched Skills, Missing Keywords, and Improvement Suggestions.

D. Resume History and Re-analysis:

A key enhancement over existing tools is ResuMind's ability to store analysis results persistently in Puter's KV store. Users can return to any previous resume, view historical analysis results, and trigger a fresh analysis against a different job description without re-uploading the file. This enables a genuine iterative improvement workflow—users can edit their resume based on suggestions and immediately re-test it against the same or a new job posting.

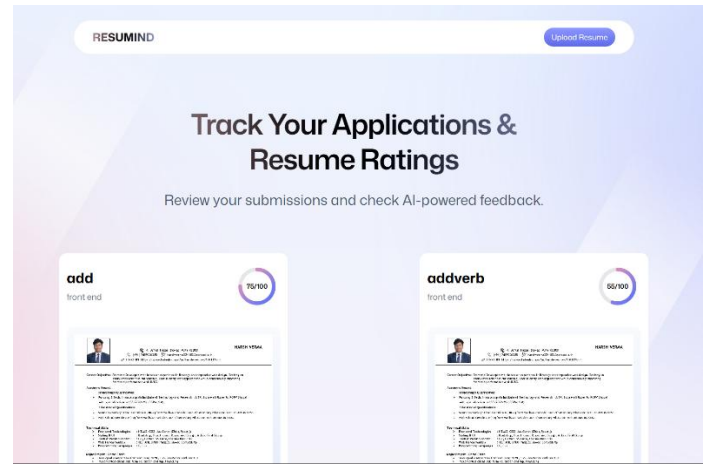


Fig. 6. Resume history and re-analysis feature.

TABLE 1: FEATURE SUMMARY

Feature	Description	Impact
ATS Score Card	Color-coded numeric ATS compatibility score out of 100.	Immediate alignment insight
Matched Skills Section Matched Skills Section	Lists keywords and competencies present in both resume and job description.	Highlights resume strengths
Missing Keywords Section	Shows keywords found in the job description but absent from the resume.	Guides targeted edits
AI Improvement Suggestions	Section-specific, AI-generated actionable feedback in plain language.	Increases application success rate
Resume History & Re-analysis	Stores all resumes and past analyses; allows re-running against new job posts.	Enables iterative improvement workflow

VII. SYSTEM ARCHITECTURE AND WORKFLOW DIAGRAM

The workflow of ResuMind covers the complete user journey from initial access through authentication, resume upload, job description input, AI analysis, and result display. The user first visits the application and is prompted to sign in via Puter.js. On the dashboard, existing resumes are shown, and new uploads can be initiated. On the resume detail view, the job description is pasted and the analysis is triggered. The AI result is displayed on a dedicated results page with the ATS score and categorized feedback.

Internally, the data flow proceeds as follows: (1) PDF is uploaded to Puter cloud storage; (2) PDF text is extracted client-side on analysis trigger; (3) a structured AI prompt is constructed incorporating the resume text and job description; (4) the prompt is sent to the Puter.js AI API which routes it to a large language model; (5) the JSON response is parsed and validated; (6) the result is saved to Puter KV and rendered in the results UI. This pipeline completes in approximately 5–15 seconds depending on network conditions and AI model response time.

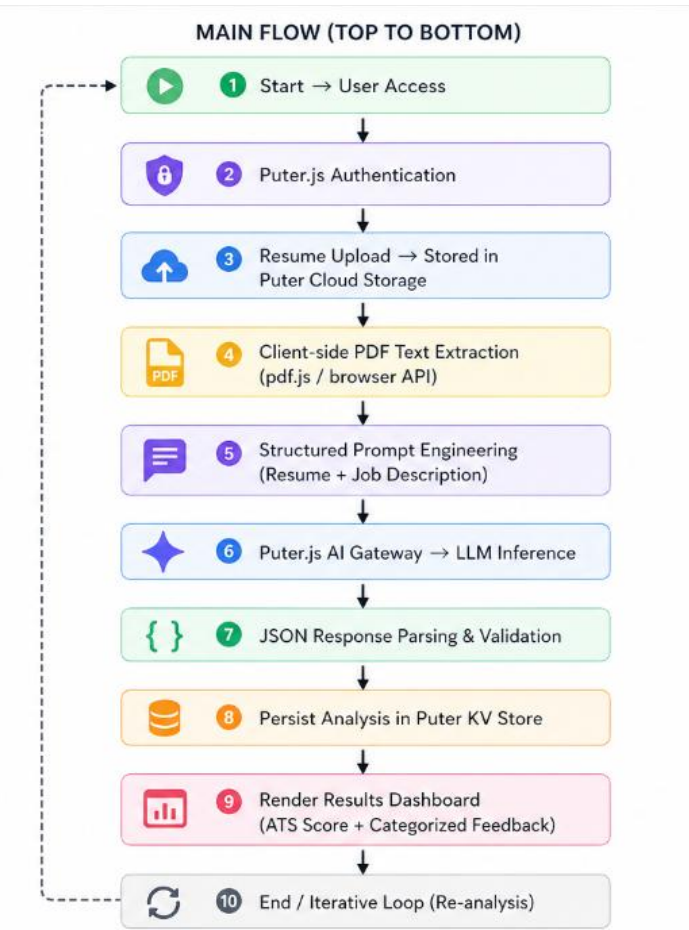


Fig. 6. Workflow Diagram of ResuMind.



Fig. 6. Layered Architecture Diagram of ResuMind.

VIII. TESTING AND EVALUATION

A. Unit Testing

Individual components and utility functions were tested in isolation. Tests covered PDF text extraction accuracy, prompt construction correctness, JSON response parsing, and React component rendering. Each module was verified to work correctly before integration into the complete application flow.

B. Integration Testing

End-to-end integration tests validated that Puter.js authentication, file storage, KV persistence, and AI API calls function correctly together. A simulated user session—sign in, upload, analyze, retrieve results—was run repeatedly to confirm that data flows without errors across all system boundaries.

C. User Acceptance Testing

Test users uploaded their actual resumes against real job postings from major job boards. Feedback was collected on the relevance of matched skills, the accuracy of missing keyword identification, and the usefulness of AI improvement suggestions. Users reported that the ATS score and sectioned feedback were actionable and significantly clearer than generic resume advice found on job-search websites.

D. Performance and Security Testing

Performance testing confirmed that the application processes large PDF files (up to 5MB) and returns AI results within acceptable time limits under normal network conditions. Security testing verified that resume files and analysis results remain private to the authenticated Puter user, with no data accessible across different user sessions.

IX. CONCLUSION

ResuMind demonstrates that a fully functional, AI-powered resume analysis platform can be built without any backend infrastructure. By combining React 19, React Router v7, TypeScript, Tailwind CSS, and Puter.js, the project delivers a practical, free tool that provides job seekers with real, job-specific feedback on their resumes in seconds.

The platform addresses a genuine and widespread problem—the opacity of ATS filtering and the absence of personalized resume feedback—and solves it with a clean, serverless, client-side application. The use of Puter.js as the platform layer removes the usual complexity of authentication, file storage, and AI integration, resulting in a codebase that is lean, maintainable, and easily extensible.

The project establishes that modern browser capabilities and client-side AI APIs are sufficient for complex document analysis tasks that previously required significant backend infrastructure. The same architectural approach could be applied to contract analysis, academic paper review, or other document-matching use cases, making ResuMind a useful reference design for the next generation of serverless AI web applications.

X. REFERENCES

- [1] J. Thompson, "The Hidden Cost of ATS Filtering in Modern Recruitment," *Journal of Human Resources Management*, vol. 15, pp. 34–47, 2023.
- [2] M. Roberts and S. Patel, "Barriers to Job Application Success: ATS and Resume Optimization," *Career Development International*, vol. 28, no. 2, pp. 2022.

- [3] A. Singh, "Comparative Analysis of Online Resume Analysis Tools," *International Journal of Computer Applications*, vol. 185, no. 12, pp. 22–29, 2023.
- [4] T. Brown et al., "Language Models are Few-Shot Learners," *NeurIPS*, 2020.
- [5] H. Zhang et al., "AI-Based Resume Screening: Accuracy and Bias Considerations," *IEEE Transactions on Neural Networks*, vol. 34, no. 6, pp. 2901–2913, 2023.
- [6] X. Li et al., "Structured Information Extraction from Unstructured Documents Using Prompt Engineering," *ACL Findings*, 2023.
- [7] R. Nair and P. Mehra, "Serverless Frontend Architectures for Complex Web Applications," *IEEE Software*, vol. 40, no. 3, pp. 58–66, 2023.
- [8] Puter.js Documentation, "Puter JavaScript SDK," [Online]. Available: <https://docs.puter.com>
- [9] D. Norman, *The Design of Everyday Things*, Revised Edition. Basic Books, 2013.
- [10] A. Amershi et al., "Software Engineering for Machine Learning: A Case Study," *IEEE/ACM ICSE-SEIP*, pp. 291–300, 2019.
- [11] React Documentation, "React 19 Release," [Online]. Available: <https://react.dev>
- [12] React Router Documentation, "React Router v7 Framework Mode," [Online]. Available: <https://reactrouter.com/home>
- [13] Vite Documentation, "Getting Started," [Online]. Available: <https://vite.dev/guide>
- [14] TypeScript Documentation, "TypeScript Handbook," [Online]. Available: <https://www.typescriptlang.org/docs>
- [15] R. Kelleher, "How Applicant Tracking Systems Work,"

